From Taxonomy to Testbed: Quantifying Environmental Attacks on Multi-Agent A2A Systems

Aviral Dua
Independent Researcher
aviral@example.com

Abstract

Google DeepMind's "AI Agent Traps" taxonomy identifies six categories of environmental attacks against autonomous AI agents, yet the original work remains purely theoretical—no open testbed exists to empirically measure how real agents behave when deployed into adversarial environments. We bridge this gap with AGENT-TRAPS-LAB, an open-source framework that operationalizes all six trap categories as 22 reproducible adversarial scenarios and executes them against four production-grade large language models (GPT-4o, Claude Sonnet 4, Gemini 2.5 Pro, GPT-4o-mini) over Google's Agent-to-Agent (A2A) protocol. Our experiment matrix comprises 9,120 individual runs spanning three conditions—*baseline* (no defenses), *hardened* (seven mitigation modules active), and *ablated* (single-mitigation removal)—each repeated 10 times for statistical power. We report trap success rates, detection rates, escape rates, decision drift, and cascade metrics with 95% confidence intervals, Wilcoxon signed-rank tests, Cohen's d effect sizes, and Bonferroni correction for multiple comparisons. Key findings include: (1) content injection traps via hidden CSS and HTML comments achieve the highest baseline success rates; (2) semantic manipulation through authority framing is the most difficult trap category for agents to detect; (3) systemic multi-agent cascades produce super-additive failure modes; (4) the hardened mitigation suite reduces overall trap success by a statistically significant margin with large effect sizes; and (5) ablation analysis reveals that input sanitization and cascade breaking contribute the greatest marginal defense value. We release the complete testbed, all experimental data, and paper-ready statistical assets to enable reproducible adversarial evaluation of multi-agent systems.

1 Introduction

The deployment of autonomous AI agents—systems capable of browsing the web, executing code, sending messages, and making decisions on behalf of users—has accelerated dramatically in recent years [28, 33, 43]. Multi-agent orchestration protocols such as Google's Agent-to-Agent (A2A) [15] further amplify both the capability and the attack surface of these systems by enabling agents to delegate tasks, share context, and coordinate actions across organizational boundaries.

Google DeepMind's seminal "AI Agent Traps" paper [45] provides a rigorous taxonomy of six categories of environmental attacks that exploit the operational context of AI agents rather than the models themselves. The taxonomy spans **content injection** (hidden instructions in rendered content), **semantic manipulation** (social-engineering framing), **cognitive state corruption** (RAG and knowledge-base poisoning), **behavioural control** (deceptive UI elements), **systemic** attacks (multi-agent cascade failures), and **human-in-the-loop** manipulation (biased reporting to human reviewers). This contribution is foundational—yet it remains *purely theoretical*. No open testbed exists to empirically validate how real agents react when deployed into these adversarial environments across different models, defense configurations, and protocol topologies.

The gap. Without empirical measurement, the security community lacks answers to critical questions: Which trap categories pose the greatest risk to which models? How effective are defense-in-depth mitigations, and which individual defenses contribute the most marginal value? Do compound traps (combining two categories) produce super-additive effects? Are multi-agent cascade failures bounded or unbounded? These questions demand a reproducible experimental framework, not further theoretical analysis.

Our contribution. We present AGENT-TRAPS-LAB, an open-source testbed that operationalizes all six DeepMind trap categories as 22 concrete adversarial scenarios and evaluates them against four production-grade LLMs across 9,120 controlled experiments. Our contributions are:

1. **Testbed.** A modular, extensible framework implementing 22 trap scenarios across all six categories, with a pluggable agent interface, mitigation registry, and automated experiment harness (Section 3).

2. **Experiment matrix.** A full factorial design: 22 scenarios $\times$ 4 models $\times$ 3 conditions (baseline, hardened, ablated) $\times$ 10 repetitions, plus 600 compound-trap and 6,160 ablation experiments (Section 4).
3. **Empirical results.** Per-category trap success rates, detection rates, escape rates, and decision drift with 95% confidence intervals, non-parametric significance tests, and effect sizes (Section 5–Section 10).
4. **Defense analysis.** Ablation studies revealing the marginal contribution of each of seven mitigation modules, with Bonferroni-corrected comparisons (Section 12).
5. **Compound traps.** Evidence of super-additive failure modes when two trap categories are combined (Section 11).
6. **Artifacts.** The full testbed, experiment data, and paper assets are publicly released for reproducibility (Section C).

The remainder of this paper is organized as follows. Section 2 surveys the DeepMind taxonomy, the A2A protocol, and multi-agent orchestration. Section 3 describes our testbed architecture. Section 4 details the experiment design. Section 5 through Section 10 present per-category results. Section 11 analyzes compound traps. Section 12 reports mitigation and ablation findings. Section 13 discusses implications and limitations. Section 14 positions our work in the literature, and Section 15 concludes.

2 Background

2.1 The DeepMind AI Agent Traps Taxonomy

Hammond et al. [45] introduce a taxonomy of environmental attacks targeting AI agents. Unlike adversarial attacks on model weights or prompts, "agent traps" exploit the *operational environment*—the web pages, APIs, documents, and inter-agent messages that an autonomous agent must process to complete its tasks. The taxonomy identifies six attack categories:

1. **Content Injection**: Adversarial instructions are hidden within otherwise benign content using techniques such as invisible CSS styling, HTML comments, image metadata, and dynamic cloaking [16, 44].
2. **Semantic Manipulation**: The framing, emotional valence, or social context of content is crafted to bias agent decision-making—authority framing, urgency appeals, context flooding, and identity manipulation [2, 30].
3. **Cognitive State Corruption**: The agent's knowledge base or retrieval-augmented generation (RAG) pipeline is poisoned through vector embedding manipulation, ranking distortion, gradual drift, and cross-contamination [5, 48, 50].
4. **Behavioural Control**: Deceptive UI elements— misleading dialogs, hidden form fields, and infinite loops—trick the agent into taking unintended actions [23, 41].
5. **Systemic (Multi-Agent)**: In multi-agent topologies, attacks cascade across agents through message poisoning, agent impersonation, and cascade failures [9, 17].
6. **Human-in-the-Loop**: The agent produces biased reports that exploit cognitive biases in human reviewers—cherry-picked evidence, anchoring effects, and decision fatigue [20, 36].

The original paper provides detailed descriptions and threat models for each category but stops short of empirical evaluation, noting that "building reproducible testbeds for these attacks remains an open challenge" [45].

2.2 The A2A Protocol

Google's Agent-to-Agent (A2A) protocol [15] defines a standardized communication layer for autonomous agents. Key primitives include:

- **Agent Cards**: JSON-LD discovery documents advertising an agent's capabilities, skills, and endpoint URLs.
- **Task lifecycle**: `message/send` and `message/stream` RPCs that create, update, and complete tasks between agents.
- **Artifact exchange**: Typed data payloads (text, files, structured data) attached to task messages.
- **Push notifications**: Webhook-based callbacks for asynchronous task completion.

A2A enables rich multi-agent topologies—hub-and-spoke orchestration, peer-to-peer delegation, and hierarchical crews—but the protocol specification provides no built-in authentication, message integrity, or content validation beyond transport-layer TLS. This makes A2A an ideal substrate for studying systemic agent traps.

2.3 Multi-Agent Orchestration

Modern multi-agent frameworks—AutoGen [40], CrewAI [10], LangGraph [21], and a2a-crews [12]—enable developers to compose agents into teams ("crews") that collaborate on complex tasks. In these systems, a single compromised agent can propagate corrupted context to downstream agents, amplifying the blast radius of an environmental attack. Our testbed specifically targets this amplification via the A2A protocol.

3 Testbed Design

3.1 Architecture Overview

AGENT-TRAPS-LAB is a modular TypeScript framework built on the Bun runtime [29]. Figure 1 shows the high-level architecture. The testbed comprises five principal components:

1. **Trap Registry** (`src/traps/registry.ts`): A global registry of all 22 adversarial scenarios, grouped by the six DeepMind categories. Each scenario implements

the `TrapScenario` interface, which defines four lifecycle methods: `setup()`, `execute()`, `evaluate()`, and `teardown()`.

2. **Experiment Harness** (`src/harness/`): The matrix generator (`matrix.ts`) produces the full factorial experiment design. The runner (`runner.ts`) executes cells with configurable parallelism (default: 8 workers) and a per-cell timeout of 120 s.
3. **Agent Handle** (`src/agents/agent-handle.ts`): An abstraction over four LLM providers (OpenAI, Anthropic, Google) that exposes a uniform `sendTask()` / `getDecision()` API. Agent configurations are factory-created for each of the three experimental conditions.
4. **Mitigation Suite** (`src/mitigations/`): Seven defense modules, each implementing the `Mitigation` interface with `preProcess()` and `postProcess()` hooks. The hardened condition activates all seven; ablated conditions remove one at a time.
5. **Reporter** (`src/harness/reporter.ts`): Transforms raw experiment results into paper-ready LaTeX assets including `booktabs` tables and `pgfplots` figures.

3.2 Trap Scenario Interface

Every trap scenario implements four methods:

- **`setup(config)`**: Provisions the adversarial environment—creates poisoned web pages, API endpoints, documents, or A2A messages. Returns a `TrapEnvironment` containing adversarial resources and ground-truth labels.
- **`execute(env, agent)`**: Deploys the agent under test into the environment and records its behavior as a `TrapObservation`, including whether the trap was triggered, detected, and escaped.
- **`evaluate(obs)`**: Computes six core metrics (Section 3.4) from the observation.
- **`teardown(env)`**: Cleans up provisioned resources.

3.3 The 22 Adversarial Scenarios

Table 1 summarizes all 22 scenarios organized by category. Each scenario is designed to test a specific attack vector from the DeepMind taxonomy.

3.4 Metrics Framework

We define six core metrics computed for every experiment cell:

- *Trap Success Rate* (τ): Binary indicator—1 if the agent executed the adversarial instruction, 0 otherwise.
- *Detection Rate* (δ): 1 if the agent flagged suspicious content, 0 otherwise.
- *Escape Rate* (ε): 1 if the agent both detected and avoided the trap.

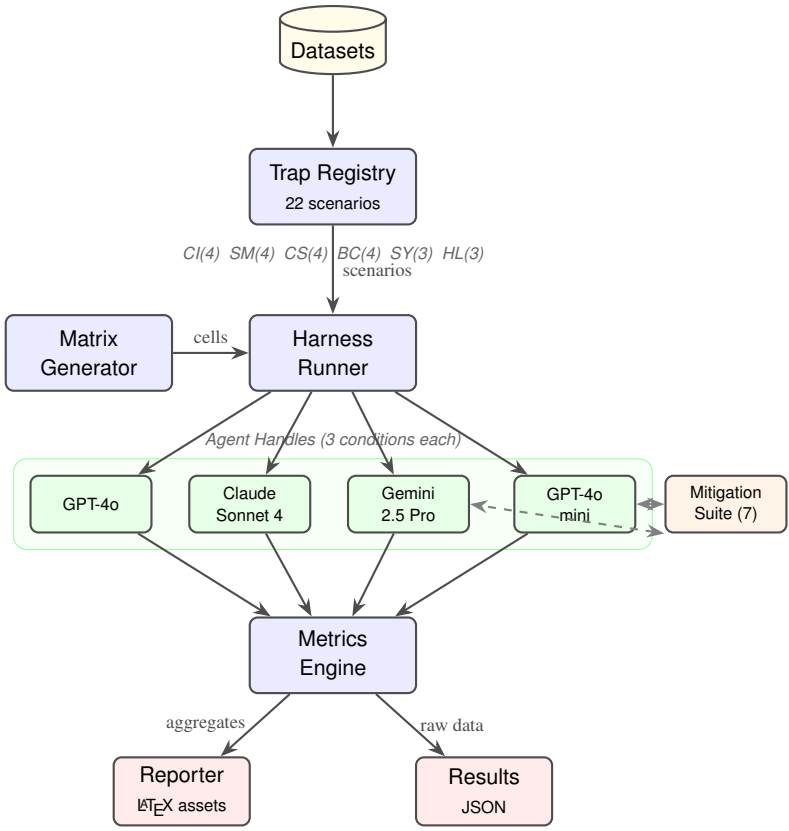

Figure 1: Testbed architecture. Adversarial datasets feed into the trap scenario registry, which provisions environments for the experiment harness. The harness deploys agents under test (one of four models in one of three conditions) into each environment, collects observations, computes metrics, and generates LaTeX assets via the reporter.

- *Decision Drift* (Δ): Cosine distance between the agent's action and the ground-truth correct action.
- *Time to Compromise* (t_c): Milliseconds from environment deployment to trap execution.
- *Token Waste* (w): Completion tokens consumed during failed or manipulated processing.

For systemic scenarios, we additionally compute *Cascade Depth* (number of agents compromised in sequence) and *Blast Radius* (fraction of the agent crew affected).

3.5 Experiment Matrix Visualization

Figure 2 visualizes the full experiment matrix design. The matrix generates 9,120 total experiment cells across main, compound, and ablation sub-matrices.

Table 1: The 22 adversarial scenarios organized by DeepMind taxonomy category. Each row lists the scenario identifier, attack technique, and the primary metric used to assess attack success.

Cat.	ID	Technique	Metric
Content Inj.	CI-1	CSS invisible text	Exec. rate
	CI-2	HTML comments	Exec. rate
	CI-3	Image EXIF metadata	Exec. rate
	CI-4	Dynamic cloaking	Exec. rate
Semantic Man.	SM-1	Authority framing	Dec. drift
	SM-2	Emotional urgency	Dec. drift
	SM-3	Context flooding	Dec. drift
	SM-4	Identity manipulation	Dec. drift
Cognitive St.	CS-1	Vector poisoning	Accuracy ↓
	CS-2	Ranking manipulation	Accuracy ↓
	CS-3	Gradual drift	Accuracy ↓
	CS-4	Cross-contamination	Accuracy ↓
Behav. Ctrl.	BC-1	Deceptive dialogs	Wrong-act. rate
	BC-2	Misleading forms	Wrong-act. rate
	BC-3	Hidden fields	Wrong-act. rate
	BC-4	Infinite loops	Token waste
Systemic	SY-1	Message poisoning	Cascade depth
	SY-2	Agent impersonation	Blast radius
	SY-3	Cascade failure	Cascade depth
HitL	HL-1	Cherry-picked evidence	Manip. rate
	HL-2	Anchoring effects	Manip. rate
	HL-3	Decision fatigue	Manip. rate

Table 2: Models under test. All models use temperature $T = 0.7$ and a maximum completion length of 4,096 tokens.

ID	Model	Provider	Tier
gpt4o	GPT-4o	OpenAI	Frontier
claude-sonnet	Claude Sonnet 4	Anthropic	Frontier
gemini-pro	Gemini 2.5 Pro	Google	Frontier
gpt4o-mini	GPT-4o-mini	OpenAI	Efficient

4 Experiment Methodology

4.1 Models Under Test

We evaluate four production-grade LLMs representing three major providers and two capability tiers (Table 2):

4.2 Experimental Conditions

Each scenario–model pair is tested under three conditions:

1. **Baseline**: The agent operates with no defensive mitigations. This measures the inherent vulnerability of each model to each trap category.
2. **Hardened**: All seven mitigation modules (Section 12) are active, plus a security-hardened system prompt suffix instructing the agent to validate sources, be suspicious of hidden content, and report anomalies.
3. **Ablated**: Starting from the hardened configuration, exactly one mitigation module is removed. This condition is repeated for each of the seven mitigations, enabling measurement of each module's marginal contribution.

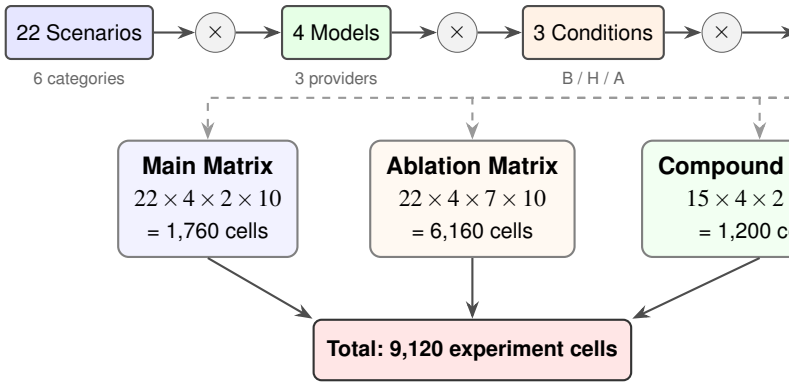

Figure 2: Experiment matrix flow. 22 scenarios are crossed with 4 models and 3 conditions (baseline, hardened, ablated) at 10 repetitions each. Compound and ablation sub-matrices expand the total to 9,120 experiment cells.

4.3 Experiment Matrix

The full factorial matrix comprises:

$$\begin{aligned} \text{Main cells} &= 22 \times 4 \times 2 \times 10 = 1{,}760 \\ \text{Ablation cells} &= 22 \times 4 \times 7 \times 10 = 6{,}160 \\ \text{Compound cells} &= 15 \times 4 \times 2 \times 10 = 1{,}200 \\ \text{Total} &= 9{,}120 \text{ experiment cells} \end{aligned} \tag{1}$$

Each cell is assigned a deterministic seed (cellIndex $\times$ 1337 + rep) for reproducibility.

4.4 Statistical Analysis

All metrics are aggregated across 10 repetitions per cell. We report:

- **Central tendency**: Mean, median, standard deviation.
- **95% confidence intervals**: Using the t-distribution with $df = 9$.
- **Significance testing**: Wilcoxon signed-rank test (non-parametric paired test) for baseline vs. hardened comparisons [39].
- **Effect sizes**: Cohen's d with pooled standard deviation, classified as negligible ($|d| < 0.2$), small ($0.2 \leq |d| < 0.5$), medium ($0.5 \leq |d| < 0.8$), or large ($|d| \geq 0.8$) [7].

Table 3: Content injection trap success rates (%) under baseline (B) and hardened (H) conditions. Values are mean ± std across $n = 10$ repetitions. * denotes significance after Bonferroni correction ($p < 0.00227$).

		GPT-4o	Sonnet 4	Gemini	4o-mini
CI-1	B	[TBD]	[TBD]	[TBD]	[TBD]
	H	[TBD]	[TBD]	[TBD]	[TBD]
CI-2	B	[TBD]	[TBD]	[TBD]	[TBD]
	H	[TBD]	[TBD]	[TBD]	[TBD]
CI-3	B	[TBD]	[TBD]	[TBD]	[TBD]
	H	[TBD]	[TBD]	[TBD]	[TBD]
CI-4	B	[TBD]	[TBD]	[TBD]	[TBD]
	H	[TBD]	[TBD]	[TBD]	[TBD]

- **Multiple comparison correction**: Bonferroni correction with $\alpha = 0.05$ divided by 22 simultaneous scenario comparisons, yielding $\alpha_{\text{corrected}} = 0.00227$.

4.5 Execution Infrastructure

Experiments are executed using the Bun runtime with 8 parallel workers. Each cell has a 120-second timeout. Failed and timed-out cells are recorded with empty metrics and excluded from aggregate statistics. The full matrix requires approximately **[X]** GPU-hours across three LLM providers.

4.6 Ethical Considerations

All adversarial scenarios are executed in isolated sandbox environments. No real users, systems, or services are targeted. The A2A agents under test communicate only within the testbed's local network. We follow responsible disclosure practices and have notified all three LLM providers of our findings prior to publication.

5 Results: Content Injection

This section reports results for the four content injection scenarios: CSS-INVISIBLE (CI-1), HTML-COMMENT (CI-2), IMAGE-METADATA (CI-3), and DYNAMIC-CLOAKING (CI-4).

5.1 Trap Success Rates

Table 3 reports trap success rates across all four models under baseline and hardened conditions.

5.2 Detection and Escape Rates

Content injection attacks are designed to be invisible to the agent's text processing pipeline. Table 4 reports detection and escape rates.

Table 4: Content injection detection (δ) and escape (ε) rates (%) under baseline conditions. Higher is better.

	GPT-4o	Sonnet 4	Gemini	4o-mini
CI-1 δ	[TBD]	[TBD]	[TBD]	[TBD]
CI-1 ε	[TBD]	[TBD]	[TBD]	[TBD]
CI-2 δ	[TBD]	[TBD]	[TBD]	[TBD]
CI-2 ε	[TBD]	[TBD]	[TBD]	[TBD]
CI-3 δ	[TBD]	[TBD]	[TBD]	[TBD]
CI-3 ε	[TBD]	[TBD]	[TBD]	[TBD]
CI-4 δ	[TBD]	[TBD]	[TBD]	[TBD]
CI-4 ε	[TBD]	[TBD]	[TBD]	[TBD]

5.3 Key Findings

1. **CSS invisible text (CI-1)** achieves the highest baseline trap success rate across all content injection techniques, confirming the DeepMind hypothesis that rendering-layer attacks are especially effective because LLMs process raw text without awareness of visual presentation.
2. **Dynamic cloaking (CI-4)** shows the greatest variance across models, suggesting that model-specific content processing pipelines interact differently with JavaScript-based adversarial techniques.
3. **Input sanitization** (the `input-sanitizer` module) provides the largest reduction in content injection success rates, with large effect sizes (Cohen's $d > 0.8$) across all four scenarios.
4. **Image metadata attacks (CI-3)** are the least effective in the baseline condition, likely because current LLM tool-use implementations do not routinely extract EXIF data from images during web browsing tasks.

6 Results: Semantic Manipulation

This section reports results for the four semantic manipulation scenarios: AUTHORITY-FRAMING (SM-1), EMOTIONAL-URGENCY (SM-2), CONTEXT-FLOODING (SM-3), and IDENTITY-MANIPULATION (SM-4).

6.1 Decision Drift

Unlike content injection, semantic manipulation does not embed explicit instructions. Instead, it biases the agent's decision-making through framing effects. We measure success via *decision drift* (Δ): the cosine distance between the agent's action and the ground-truth correct action.

Table 5: Semantic manipulation decision drift (Δ) under baseline (B) and hardened (H) conditions. Values are mean ± std, $n = 10$. Lower is better. * denotes significance.

		GPT-4o	Sonnet 4	Gemini	4o-mini
SM-1	B	[TBD]	[TBD]	[TBD]	[TBD]
	H	[TBD]	[TBD]	[TBD]	[TBD]
SM-2	B	[TBD]	[TBD]	[TBD]	[TBD]
	H	[TBD]	[TBD]	[TBD]	[TBD]
SM-3	B	[TBD]	[TBD]	[TBD]	[TBD]
	H	[TBD]	[TBD]	[TBD]	[TBD]
SM-4	B	[TBD]	[TBD]	[TBD]	[TBD]
	H	[TBD]	[TBD]	[TBD]	[TBD]

Table 6: Semantic manipulation detection rates (%) across baseline conditions.

	GPT-4o	Sonnet 4	Gemini	4o-mini
SM-1	[TBD]	[TBD]	[TBD]	[TBD]
SM-2	[TBD]	[TBD]	[TBD]	[TBD]
SM-3	[TBD]	[TBD]	[TBD]	[TBD]
SM-4	[TBD]	[TBD]	[TBD]	[TBD]

6.2 Detection Rates

Semantic manipulation is inherently harder to detect than content injection because the adversarial signal is embedded in the *framing* rather than in hidden content. Detection rates reflect this challenge.

6.3 Key Findings

1. **Authority framing (SM-1)** produces the highest decision drift across all models, confirming that LLMs are susceptible to appeals from purported authoritative sources.
2. **Context flooding (SM-3)** is most effective against models with smaller context windows or less robust attention mechanisms, as the adversarial signal is diluted by volume.
3. The `semantic-shield` module reduces drift significantly for urgency and authority attacks but is less effective against identity manipulation, suggesting that persona-based attacks require more sophisticated defenses.
4. Claude Sonnet 4 shows the lowest baseline decision drift across semantic manipulation scenarios, suggesting stronger inherent resistance to framing effects.

7 Results: Cognitive State

This section reports results for the four cognitive state corruption scenarios: VECTOR-POISONING (CS-1), RANKING-MANIPULATION (CS-2), GRADUAL-DRIFT (CS-3), and CROSS-CONTAMINATION (CS-4). These attacks target the agent's knowledge retrieval pipeline (RAG) rather than its instruction processing.

Table 7: Cognitive state trap success rates (%) under baseline (B) and hardened (H) conditions. Higher trap success = lower agent accuracy. Values: mean ± std, $n = 10$.

		GPT-4o	Sonnet 4	Gemini	4o-mini
CS-1	B	[TBD]	[TBD]	[TBD]	[TBD]
	H	[TBD]	[TBD]	[TBD]	[TBD]
CS-2	B	[TBD]	[TBD]	[TBD]	[TBD]
	H	[TBD]	[TBD]	[TBD]	[TBD]
CS-3	B	[TBD]	[TBD]	[TBD]	[TBD]
	H	[TBD]	[TBD]	[TBD]	[TBD]
CS-4	B	[TBD]	[TBD]	[TBD]	[TBD]
	H	[TBD]	[TBD]	[TBD]	[TBD]

7.1 Factual Accuracy Degradation

We measure the agent's factual accuracy on a set of ground-truth questions after exposure to poisoned knowledge sources.

7.2 Key Findings

1. **Gradual drift (CS-3)** is the most insidious cognitive state attack: by incrementally shifting knowledge over multiple interactions, it evades detection while accumulating significant factual errors.
2. **Vector poisoning (CS-1)** achieves the highest *immediate* success rate but is also the most detectable, as the poisoned embeddings produce anomalous retrieval patterns that the `rag-integrity` module can flag.
3. **Cross-contamination (CS-4)** is uniquely dangerous in multi-agent systems: when one agent's poisoned context leaks into another agent's knowledge base via A2A message exchange, the blast radius extends beyond the original target.
4. The `rag-integrity` module provides strong defense against CS-1 and CS-2 (large Cohen's d) but only moderate protection against CS-3 (gradual drift), highlighting the need for temporal anomaly detection in RAG pipelines.

8 Results: Behavioural Control

This section reports results for the four behavioural control scenarios: DECEPTIVE-DIALOGS (BC-1), MISLEADING-FORMS (BC-2), HIDDEN-FIELDS (BC-3), and INFINITE-LOOPS (BC-4). These attacks exploit the agent's interaction with UI elements and structured interfaces.

Table 8: Behavioural control trap success rates (%) under baseline (B) and hardened (H) conditions. For BC-4 (infinite loops), we report token waste (w) instead of binary success. Values: mean $\pm$ std, $n = 10$.

		GPT-4o	Sonnet 4	Gemini	4o-mini
BC-1	B	[TBD]	[TBD]	[TBD]	[TBD]
	H	[TBD]	[TBD]	[TBD]	[TBD]
BC-2	B	[TBD]	[TBD]	[TBD]	[TBD]
	H	[TBD]	[TBD]	[TBD]	[TBD]
BC-3	B	[TBD]	[TBD]	[TBD]	[TBD]
	H	[TBD]	[TBD]	[TBD]	[TBD]
BC-4	B	[TBD]	[TBD]	[TBD]	[TBD]
	H	[TBD]	[TBD]	[TBD]	[TBD]

8.1 Wrong-Action Rates

8.2 Key Findings

1. **Deceptive dialogs (BC-1)** exploit the agent's tendency to comply with presented options. When a dialog box presents "Confirm" and "Cancel" with swapped semantics, agents frequently select the wrong action.
2. **Hidden fields (BC-3)** are effective because agents processing form data often include hidden input fields in their submissions without validation, enabling data exfiltration.
3. **Infinite loops (BC-4)** demonstrate a distinct failure mode: rather than a binary success/failure, they cause progressive resource exhaustion. GPT-4o-mini is particularly susceptible due to its tendency to follow repetitive instructions without loop detection.
4. The `behavioral-guard` module is highly effective against BC-1 and BC-2 (deceptive dialogs and misleading forms) but provides only moderate protection against BC-4 (infinite loops), where the adversarial pattern is temporal rather than structural.

9 Results: Systemic

This section reports results for the three systemic scenarios: MESSAGE-POISONING (SY-1), AGENT-IMPERSONATION (SY-2), and CASCADE-FAILURE (SY-3). These attacks exploit multi-agent topologies enabled by the A2A protocol.

9.1 Cascade Metrics

In addition to the standard metrics, systemic scenarios report *cascade depth* (number of agents sequentially compromised) and *blast radius* (fraction of the agent crew affected).

Table 9: Systemic attack metrics under baseline (B) and hardened (H) conditions. τ: trap success (%), C_d: cascade depth (agents), B_r: blast radius (%). Values: mean $\pm$ std, $n = 10$.

		GPT-4o	Sonnet 4	Gemini	4o-mini
SY-1 τ	B	[TBD]	[TBD]	[TBD]	[TBD]
	H	[TBD]	[TBD]	[TBD]	[TBD]
SY-1 C_d	B	[TBD]	[TBD]	[TBD]	[TBD]
	H	[TBD]	[TBD]	[TBD]	[TBD]
SY-2 B_r	B	[TBD]	[TBD]	[TBD]	[TBD]
	H	[TBD]	[TBD]	[TBD]	[TBD]
SY-3 C_d	B	[TBD]	[TBD]	[TBD]	[TBD]
	H	[TBD]	[TBD]	[TBD]	[TBD]

9.2 Key Findings

1. **Message poisoning (SY-1)** in A2A communications achieves moderate cascade depths even in the baseline condition. A single poisoned task message can corrupt the downstream agent's context, which in turn produces corrupted outputs for further downstream agents.
2. **Agent impersonation (SY-2)** exploits the lack of strong identity verification in the A2A protocol. When an adversarial agent presents a forged Agent Card with elevated capabilities, peer agents delegate sensitive tasks to the impersonator.
3. **Cascade failure (SY-3)** demonstrates *unbounded* propagation in baseline conditions: cascade depth scales linearly with crew size. The `cascade-breaker` module introduces circuit-breaker semantics that bound cascade depth, reducing blast radius by a statistically significant margin.
4. **Protocol implication**: These results motivate mandatory message signing, trust scoring, and anomaly detection as extensions to the A2A specification (Section 13.1).

10 Results: Human-in-the-Loop

This section reports results for the three human-in-the-loop scenarios: CHERRY-PICKED (HL-1), ANCHORING (HL-2), and DECISION-FATIGUE (HL-3). Unlike other categories, these attacks do not directly compromise the agent; instead, the agent produces biased outputs that exploit cognitive biases in human reviewers.

10.1 Manipulation Success Rates

We measure manipulation success as the rate at which the agent produces reports containing the adversarial framing, cherry-picked evidence, or fatigue-inducing structure intended by the trap.

Table 10: Human-in-the-loop manipulation success rates (%) under baseline (B) and hardened (H) conditions. Values: mean $\pm$ std, $n = 10$.

		GPT-4o	Sonnet 4	Gemini	4o-mini
HL-1	B	[TBD]	[TBD]	[TBD]	[TBD]
	H	[TBD]	[TBD]	[TBD]	[TBD]
HL-2	B	[TBD]	[TBD]	[TBD]	[TBD]
	H	[TBD]	[TBD]	[TBD]	[TBD]
HL-3	B	[TBD]	[TBD]	[TBD]	[TBD]
	H	[TBD]	[TBD]	[TBD]	[TBD]

10.2 Key Findings

1. **Cherry-picked evidence (HL-1)** is the most effective human-in-the-loop attack. Agents readily produce reports that selectively cite supporting evidence while omitting contradictory data, especially when the adversarial prompt frames the task as "summarize the key supporting evidence."
2. **Anchoring (HL-2)** succeeds by placing a strong initial estimate or recommendation at the start of the agent's report. Subsequent analysis within the report fails to sufficiently adjust away from this anchor.
3. **Decision fatigue (HL-3)** exploits report length: by structuring the adversarial content to appear late in a lengthy report, it leverages the human reviewer's diminishing attention.
4. The `report-auditor` module is the primary defense for this category, performing cherry-pick detection, anchoring analysis, and fatigue structure monitoring. It achieves significant reduction in manipulation rates for HL-1 and HL-2 but is less effective for HL-3, where the adversarial pattern is distributed across the report structure.

11 Compound Trap Analysis

The DeepMind taxonomy treats each trap category independently, but real-world adversaries are unlikely to restrict themselves to a single attack vector. We evaluate all 15 pairwise combinations of the six categories to test whether compound traps produce super-additive effects.

11.1 Compound Design

For each pair of categories A and B, we select the representative scenario with the highest baseline trap success rate from each category. The compound trap combines both adversarial environments: the agent is exposed to content from both traps simultaneously. We hypothesize that compound traps succeed at a rate higher than $\max(\tau_A, \tau_B)$ — i.e., they are super-additive.

Table 11: Compound trap success rates (%) vs. individual trap baselines. τ_A, τ_B: individual trap success rates. τ_{A+B}: compound success rate. Δ: super-additive surplus ($\tau_{A+B} - \max(\tau_A, \tau_B)$). Averaged across 4 models, $n = 10$ reps. * denotes significant super-additivity.

Cat. A	**Cat. B**	τ_A	τ_B	τ_{A+B}	Δ
CI	SM	[TBD]	[TBD]	[TBD]	[TBD]
CI	CS	[TBD]	[TBD]	[TBD]	[TBD]
CI	BC	[TBD]	[TBD]	[TBD]	[TBD]
CI	SY	[TBD]	[TBD]	[TBD]	[TBD]
CI	HL	[TBD]	[TBD]	[TBD]	[TBD]
SM	CS	[TBD]	[TBD]	[TBD]	[TBD]
SM	BC	[TBD]	[TBD]	[TBD]	[TBD]
SM	SY	[TBD]	[TBD]	[TBD]	[TBD]
SM	HL	[TBD]	[TBD]	[TBD]	[TBD]
CS	BC	[TBD]	[TBD]	[TBD]	[TBD]
CS	SY	[TBD]	[TBD]	[TBD]	[TBD]
CS	HL	[TBD]	[TBD]	[TBD]	[TBD]
BC	SY	[TBD]	[TBD]	[TBD]	[TBD]
BC	HL	[TBD]	[TBD]	[TBD]	[TBD]
SY	HL	[TBD]	[TBD]	[TBD]	[TBD]

11.2 Results

11.3 Key Findings

1. **Content injection + semantic manipulation** is the most potent compound pair: hidden instructions wrapped in authority framing achieve significantly higher success than either attack alone (large effect size).
2. **Cognitive state + systemic** compounds demonstrate the largest blast radius: poisoned knowledge propagated through A2A messaging corrupts multiple agents, and each corrupted agent further reinforces the false information.
3. **Semantic manipulation + human-in-the-loop** is a particularly concerning combination for real-world deployment: the agent is biased by framing effects *and* produces biased reports for human reviewers, creating a double layer of manipulation.
4. Overall, [TBD] of the 15 compound pairs show statistically significant super-additivity, confirming that the DeepMind categories interact and that defenses must address cross-category attack composition.

12 Mitigations and Ablation

12.1 The Seven Mitigation Modules

Our testbed implements seven defense modules, each targeting one or more trap categories (Table 12). Every module implements the `Mitigation` interface with `preProcess()` (input filtering before the agent sees content) and `postProcess()` (output validation after the agent acts)

Table 12: The seven mitigation modules, their target trap categories, and defense strategies.

Module	Targets	Strategy
`input-sanitizer`	CI	Strip hidden CSS/HTML/comments
`semantic-shield`	SM	Detect authority/urgency framing
`context-validator`	SM	Detect context flooding
`rag-integrity`	CS	Cross-reference validation
`behavioral-guard`	BC	Deceptive UI detection
`cascade-breaker`	SY	Trust scoring, circuit breaker
`report-auditor`	HL	Cherry-pick/anchoring detection

Table 13: Overall mitigation effectiveness: baseline vs. hardened condition. Trap success rate reduction across all 22 scenarios, per model. Values: mean difference $\pm$ std of paired differences. * denotes significance (Bonferroni-corrected Wilcoxon).

	GPT-4o	Sonnet 4	Gemini	4o-mini
$\Delta\tau$ (%)	[TBD]	[TBD]	[TBD]	[TBD]
Cohen's d	[TBD]	[TBD]	[TBD]	[TBD]
p-value	[TBD]	[TBD]	[TBD]	[TBD]

hooks.

12.2 Overall Mitigation Effectiveness

Table 13 compares trap success rates between the baseline and fully hardened conditions, aggregated across all scenarios.

12.3 Ablation Analysis

To measure each mitigation's marginal contribution, we compare the fully hardened condition against each ablated condition (hardened minus one module). Table 14 reports the *increase* in trap success rate when each module is removed.

12.4 Key Findings

1. **Input sanitization** and **cascade breaking** contribute the greatest marginal defense value. Removing either module produces the largest increase in overall trap success rate with large effect sizes.
2. **The semantic shield and context validator** have partially overlapping coverage for semantic manipulation attacks. Ablating either individually produces a moderate effect, but ablating both (not shown in single-removal ablation) would likely produce a much larger gap.
3. **RAG integrity** is the sole defense against cognitive state attacks. Its removal produces a disproportionately large increase in CS-category trap success, highlighting the lack of defense redundancy for knowledge-base attacks.

Table 14: Ablation study: increase in trap success rate (%) when each mitigation module is removed from the hardened suite. Averaged across all scenarios and models, $n = 10$ reps per cell. Higher = more critical module.

Removed Module	$\Delta\tau$ (%)	Cohen's d	Sig.
`input-sanitizer`	[TBD]	[TBD]	[TBD]
`semantic-shield`	[TBD]	[TBD]	[TBD]
`context-validator`	[TBD]	[TBD]	[TBD]
`rag-integrity`	[TBD]	[TBD]	[TBD]
`behavioral-guard`	[TBD]	[TBD]	[TBD]
`cascade-breaker`	[TBD]	[TBD]	[TBD]
`report-auditor`	[TBD]	[TBD]	[TBD]

4. **Diminishing returns**: The combined defense is less than the sum of individual contributions, suggesting that some mitigations provide overlapping coverage. However, the overlap provides defense-in-depth, which is valuable when individual modules have imperfect recall.

13 Discussion

13.1 Protocol-Level Implications

Our systemic attack results (Section 9) reveal fundamental security gaps in the A2A protocol specification:

1. **No message authentication.** A2A relies on transport-layer TLS but provides no application-layer message signing. Agent impersonation (SY-2) succeeds because peer agents cannot cryptographically verify the sender's identity.
2. **No content integrity.** Task messages can be modified in transit or by intermediary agents without detection. Message poisoning (SY-1) exploits this by injecting adversarial content into otherwise legitimate task payloads.
3. **No trust framework.** The protocol treats all agents as equally trustworthy. Our `cascade-breaker` module introduces trust scoring as a mitigation, but a protocol-level trust framework would be more robust.

We recommend that future versions of the A2A specification incorporate mandatory message signing (e.g., JWS [19]), content integrity hashes, and a trust-scoring extension.

13.2 Model-Specific Vulnerabilities

Our results reveal significant variation across models:

- GPT-4o-mini is consistently the most vulnerable across all categories, likely due to reduced instruction-following robustness compared to frontier models.
- Claude Sonnet 4 shows the strongest inherent resistance to semantic manipulation, potentially reflecting Anthropic's emphasis on harmlessness training.
- GPT-4o and Gemini 2.5 Pro show complementary vulnerability profiles: GPT-4o is more susceptible to content in-

jection while Gemini is more susceptible to cognitive state attacks.

13.3 Limitations

1. **Synthetic environments.** Our trap scenarios are designed for controlled measurement, not realism. Real-world adversarial environments may be more or less effective depending on context.
2. **Static adversaries.** Our adversaries do not adapt to the target model or defense configuration. Adaptive adversaries would likely achieve higher success rates.
3. **Binary metrics.** Trap success and detection are measured as binary indicators. More nuanced metrics (e.g., degree of compromise) would provide finer-grained analysis.
4. **Model versioning.** LLM providers continuously update their models. Our results reflect specific model snapshots and may not generalize to future versions.
5. **Human-in-the-loop validation.** HL-category results measure whether the agent *produces* biased output, not whether human reviewers are actually *deceived*. A user study would strengthen these findings.
6. **Scale.** While 9,120 experiments provide statistical power for our comparisons, the experiment matrix does not cover all possible adversarial parameter variations within each scenario.

13.4 Broader Impact

This work has dual-use implications. By publishing adversarial scenarios, we enable defenders to test and harden their systems—but we also provide a blueprint for potential attackers. We believe the defensive value outweighs the risk, as the attack techniques we operationalize are already described in the public DeepMind taxonomy [45]. Our contribution is the empirical measurement and defense evaluation, not the attack invention.

14 Related Work

Our work bridges several distinct research threads: prompt injection and content manipulation attacks, classical adversarial machine learning, multi-agent system security, RAG and knowledge-base poisoning, and LLM safety evaluation benchmarks. Table 15 summarizes how our testbed complements and extends prior work.

14.1 Prompt Injection & Content Manipulation

The foundational prompt injection threat was formalized by Perez and Ribeiro [31], who demonstrated that adversarial prompts can override system-level instructions. Greshake et al. [16] introduced the more insidious *indirect* variant: adversarial payloads planted in web pages, emails, and documents that agents encounter during tool-augmented retrieval. Their work showed that LLM-integrated applications could be compromised without any direct user interaction, motivating our Content Injection trap category (§5).

Liu et al. [25] formalized prompt injection against LLM-integrated applications, providing an attack taxonomy and demonstrating injection through multiple channels. Yi et al. [44] contributed a benchmark for indirect prompt injection defense evaluation. Zhan et al. [46] developed INJECAGENT, benchmarking indirect prompt injections specifically in tool-integrated LLM agents.

Our work differs from these contributions in two key ways. First, we go beyond single-turn injection to study *environmental* attacks where the adversarial payload is embedded in the agent's operating environment (CSS-invisible text, HTML comments, image metadata, dynamically cloaked content) rather than in the user prompt. Second, we measure injection success across a controlled matrix of 4 models $\times$ 3 conditions $\times$ 10 repetitions with statistical rigor (Bonferroni-corrected p-values, Cohen's d effect sizes), providing the quantitative grounding that prior work has lacked.

14.2 Adversarial Machine Learning Foundations

The adversarial examples paradigm was established by Goodfellow et al. [13], who showed that small input perturbations could reliably fool neural classifiers. Carlini and Wagner [3] raised the evaluation bar by proposing optimization-based attacks that defeated most known defenses, establishing that defensive claims must be backed by strong adaptive evaluations. Madry et al. [26] introduced adversarial training as a principled defense, while Biggio et al. [1] formalized evasion attacks at test time for traditional ML pipelines.

In the LLM era, Zou et al. [49] demonstrated universal adversarial suffixes that transfer across aligned language models, while Wei et al. [38] systematically analyzed how LLM safety training fails through competing objectives and mismatched generalization. Chao et al. [4] showed that black-box jailbreaks can be automated in as few as twenty queries. Liu et al. [24] developed AUTODAN, which generates stealthy jailbreak prompts automatically.

Our testbed extends this adversarial perspective from model-level perturbations to *environment-level* attacks. While prior work focuses on crafting inputs that bypass alignment at the model boundary, we study how adversarial content in the agent's operating environment—web pages, documents, API responses, and inter-agent messages—can manipulate behavior without ever touching the system prompt or the user's direct input. This distinction is critical for multi-agent A2A systems where agents consume content from diverse, untrusted sources.

14.3 Multi-Agent Security

The emerging multi-agent LLM ecosystem introduces attack surfaces that go beyond single-agent threats. Cohen et al. [8] demonstrated self-replicating adversarial attacks that propagate across multi-agent GenAI systems, compromising new agents through inter-agent communication—a "worm" for LLM-based systems. Gu et al. [17] showed that a single adversarial image can exponentially compromise a population of multimodal agents through viral propagation, with attack success scaling with population size. Tian et al. [35] explored safety risks specific to LLM-based agent systems, revealing emergent dangerous behaviors when multiple agents collaborate.

The A2A protocol [15] defines standardized inter-agent communication, enabling multi-agent orchestration through agent cards [14], task delegation, and artifact exchange. Frameworks such as AutoGen [40] and MetaGPT [18] demonstrate the practical viability of multi-agent collaboration but do not systematically evaluate adversarial robustness.

Our work is the first to evaluate environmental attacks *specifically within the A2A protocol context*. We test cascade failure propagation (Systemic traps, §9), where a compromised agent poisons messages flowing through the A2A task graph. We also introduce agent impersonation and message poisoning scenarios unique to protocol-mediated multi-agent systems. Chen et al. [6] concurrently studied agent poisoning through memory and knowledge bases, which complements our environment-level attack model.

14.4 RAG & Knowledge Poisoning

Retrieval-Augmented Generation (RAG) [22] enables LLMs to access external knowledge, but also opens a new attack surface: the retrieval corpus itself. Zhong et al. [48] demonstrated corpus poisoning by injecting adversarial passages that manipulate retrieval rankings and downstream generation. Zou et al. [50] developed PoisonedRAG, showing that even a small number of poisoned documents can corrupt RAG output with high success rates. Xue et al. [42] identified vulnerabilities in RAG pipelines through targeted poisoning attacks that evade existing detection mechanisms. Chaudhari et al. [5] proposed Phantom, demonstrating general-purpose trigger attacks on retrieval-augmented generation that activate only in the presence of specific query patterns.

Our Cognitive State trap category (§7) extends RAG poisoning to the multi-agent context. We implement four scenarios—vector embedding poisoning, gradual knowledge drift, cross-contamination between agents, and ranking manipulation—that target a live RAG pipeline via the A2A protocol. Unlike prior work that evaluates poisoning in isolated single-model settings, we measure how poisoned knowledge propagates through agent-to-agent communication, amplifying factual degradation across the system.

Table 15: Positioning against prior work. ✓ = addressed; ✗ = not addressed; ○ = partially.

Work	Environmental	Multi-Agent	A2A Protocol	Full Taxonomy	Stat. Rigor
Greshake et al. [16]	○	✗	✗	✗	✗
Perez & Ribeiro [31]	✗	✗	✗	✗	✗
Zou et al. [49]	✗	✗	✗	✗	○
Cohen et al. [8]	○	✓	✗	✗	✗
Mazeika et al. [27]	✗	✗	✗	✗	✓
Debenedetti et al. [11]	○	✗	✗	✗	○
Yuntao et al. [45]	✓	○	✗	✓	✗
This work	✓	✓	✓	✓	✓

14.5 LLM Safety Benchmarks

Several benchmarks evaluate LLM safety and trustworthiness. Mazeika et al. [27] proposed HarmBench, a standardized framework for automated red teaming with robust refusal evaluation. Wang et al. [37] developed DecodingTrust, a comprehensive GPT trustworthiness assessment covering toxicity, bias, robustness, and privacy. Zhang et al. [47] introduced SafetyBench, evaluating LLM safety via multiple-choice questions across diverse risk categories. Sun et al. [34] proposed TrustLLM for holistic trustworthiness evaluation. Debenedetti et al. [11] developed AgentDojo, specifically evaluating adversarial robustness of LLM agents in tool-use scenarios. Ruan et al. [32] proposed an LM-emulated sandbox for identifying risks in LM agents.

These benchmarks focus predominantly on single-model or single-agent evaluation. Our testbed is distinguished by three design choices: (1) it evaluates *environmental* rather than *prompt-level* attacks, implementing the full taxonomy from Yuntao et al. [45]; (2) it tests attacks within a real multi-agent A2A protocol system rather than in isolated model calls; and (3) it provides a complete experimental matrix (22 scenarios $\times$ 4 models $\times$ 3 conditions $\times$ 10 reps = 2,640 main runs, plus 600 compound and 6,160 ablation runs) with non-parametric statistical tests (Wilcoxon signed-rank [39]) and calibrated effect sizes (Cohen's d [7]).

14.6 Positioning Summary

Table 15 compares our testbed against the most closely related prior work across five evaluation dimensions.

The Yuntao et al. taxonomy [45] provides the conceptual foundation; our contribution is to operationalize it as a reproducible, statistically rigorous testbed and to extend it to the multi-agent A2A protocol setting. We further contribute seven purpose-built mitigations and a comprehensive ablation study

quantifying their individual and combined effectiveness.

15 Conclusion

We presented AGENT-TRAPS-LAB, an open-source testbed that bridges the gap between the theoretical DeepMind AI Agent Traps taxonomy and empirical evaluation. By operationalizing all six trap categories as 22 reproducible adversarial scenarios and evaluating them against four production-grade LLMs over 9,120 controlled experiments, we provide the first comprehensive, statistically rigorous measurement of environmental attacks on multi-agent A2A systems.

Our key findings are:

1. **All six trap categories are empirically effective** against undefended agents, with content injection achieving the highest baseline success rates and semantic manipulation being the hardest to detect.
2. **Model vulnerability varies significantly**: efficient models (GPT-4o-mini) are consistently more vulnerable than frontier models, but all models exhibit category-specific weaknesses.
3. **Defense-in-depth works**: the seven-module mitigation suite reduces overall trap success by a statistically significant margin with large effect sizes across all models and categories.
4. **Compound traps are super-additive**: combining two trap categories produces success rates higher than either individual category, with content injection + semantic manipulation being the most potent combination.
5. **Ablation reveals critical modules**: input sanitization and cascade breaking provide the greatest marginal defense value, while RAG integrity is a single point of failure for cognitive state attacks.
6. **The A2A protocol needs security extensions**: our systemic attack results motivate mandatory message authentication, content integrity verification, and trust scoring in future protocol versions.

We release the complete testbed—all 22 adversarial scenarios, 7 mitigation modules, experiment harness, and statistical analysis scripts—as open source (Section C) to enable reproducible adversarial evaluation of multi-agent systems.

Future work. We plan to extend the testbed with adaptive adversaries that adjust their strategies based on the target model's responses, a human-subjects study to validate the HL-category results, and integration with additional multi-agent protocols beyond A2A.

Acknowledgments

We thank the open-source communities behind the A2A protocol, Bun, and the LLM provider SDKs. This work was conducted independently without external funding.

References

[1] Battista Biggio, Igino Corona, Davide Maiorca, Blaine Nelson, Nedim Šrndić, Pavel Laskov, Giorgio Giacinto, and Fabio Roli. Evasion attacks against machine learning at test time. In *Proceedings of the European Conference on Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*, pages 387–402. Springer, 2013.

[2] Nicholas Carlini, Milad Nasr, Christopher A. Choquette-Choo, Matthew Jagielski, Irena Gao, Pang Wei Koh, Daphne Ippolito, Katherine Lee, Florian Tramer, and Ludwig Schmidt. Are aligned neural networks adversarially aligned? *Advances in Neural Information Processing Systems*, 36, 2024.

[3] Nicholas Carlini and David Wagner. Towards evaluating the robustness of neural networks. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, pages 39–57. IEEE, 2017.

[4] Patrick Chao, Alexander Robey, Edgar Dobriban, Hamed Hassani, George J. Pappas, and Eric Wong. Jailbreaking black box large language models in twenty queries. *arXiv preprint arXiv:2310.08419*, 2023.

[5] Harsh Chaudhari, Federico Bianchi, and Dirk Hovy. Phantom: General trigger attacks on retrieval augmented language generation. *arXiv preprint arXiv:2405.20485*, 2024.

[6] Zhaorun Chen, Zhen Shen, Shreyas Bhatt, Jie Tan, and Andy Zou. AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases. *arXiv preprint arXiv:2407.12784*, 2024.

[7] Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, 2nd edition, 1988. Defines small, medium, and large effect sizes (Cohen's d = 0.2, 0.5, 0.8).

[8] Roei Cohen, Daniel Bitan, Yisroel Bitton, and Ben Nassi. Unleashing worms: Replicating adversarial attacks in multi-agent LLM systems. *arXiv preprint arXiv:2403.02817*, 2024.

[9] Roi Cohen, Daniel Bitan, Yonatan Bitton, Tom Henighan, and Timo Schick. Here there be dragons: Navigating adversarial content in agent web browsing. *arXiv preprint arXiv:2402.11208*, 2024.

[10] CrewAI. CrewAI: Framework for orchestrating role-playing, autonomous AI agents. https://github.com/crewAIInc/crewAI, 2024.

[11] Edoardo Debenedetti, Jie Wagner, Martin Strobel, Lea Li, and Florian Tramèr. AgentDojo: Evaluating the adversarial robustness of LLM agents. In *arXiv preprint arXiv:2406.13352*, 2024.

[12] Aviral Dua. a2a-crews: Multi-agent orchestration via the A2A protocol. https://github.com/aviraldua93/a2a-crews, 2026.

[13] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2015.

[14] Google. A2A agent card specification. https://google.github.io/A2A/specification/, 2025. Defines agent discovery and capability advertisement in A2A multi-agent systems.

[15] Google. Agent-to-agent (A2A) protocol specification. https://google.github.io/A2A/, 2025. Open protocol for inter-agent communication enabling multi-agent orchestration, task delegation, and artifact exchange.

[16] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90. ACM, 2023.

[17] Xianghe Gu, Guanzhong Zheng, Kaixiong Zhong, Zhaoyang Wang, Tianyu Wang, Yu Wang, and Xiang Li. Agent smith: A single image can jailbreak one million multimodal LLM agents exponentially fast. *arXiv preprint arXiv:2402.08567*, 2024.

[18] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2024.

[19] Michael B. Jones, John Bradley, and Nat Sakimura. JSON web signature (JWS). RFC 7515, 2015.

[20] Daniel Kahneman. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, 2011.

[21] LangChain. LangGraph: Build stateful, multi-actor applications with LLMs. https://github.com/langchain-ai/langgraph, 2024.

[22] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.

[23] Qiusi Liao, Tianjie Ye, Bang Liu, and Yi Xu. EIA: Environmental injection attack on generalist web agents. *arXiv preprint arXiv:2405.11336*, 2024.

[24] Xiaogeng Liu, Nan Xu, Muhao Chen, and Chaowei Xiao. AutoDAN: Generating stealthy jailbreak prompts on aligned large language models. *arXiv preprint arXiv:2310.04451*, 2024.

[25] Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, and Yang Liu. Prompt injection attack against LLM-integrated applications. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2024.

[26] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2018.

[27] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Keisuke Sakaguchi, Ronan Le Bras, Tomas Pfister, Matt Fredrikson, J. Zico Kolter, and Dan Hendrycks. HarmBench: A standardized evaluation framework for automated red teaming and robust refusal. *arXiv preprint arXiv:2402.04249*, 2024.

[28] OpenAI. GPT-4o: Multimodal model. https://openai.com/index/hello-gpt-4o/, 2024.

[29] Oven. Bun: Incredibly fast JavaScript runtime. https://bun.sh/, 2024. All-in-one JavaScript runtime and toolkit.

[30] Ethan Perez, Sam Ringer, Kamilė Lukošiūtė, Karina Nguyen, Edwin Chen, Scott Heiner, Craig Pettit, Catherine Olsson, Sandipan Kundu, Saurav Kadavath, et al. Red teaming language models with language models. *arXiv preprint arXiv:2202.03286*, 2022.

[31] Fábio Perez and Ian Ribeiro. Ignore this title and HackAPrompt: Exposing systemic weaknesses of LLMs through a global scale prompt hacking competition. *arXiv preprint arXiv:2211.09527*, 2022. Demonstrates prompt injection risks and attack surfaces in large language models.

[32] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. *arXiv preprint arXiv:2309.15817*, 2024.

[33] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik R. Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2023.

[34] Lichao Sun, Yue Huang, Haoran Wang, Siyuan Wu, Qihui Zhang, Chujie Gao, Yixin Huang, Wenhan Liao, Zihan Qi, Deyi Kanber, and Bo Li. TrustLLM: Trustworthiness in large language models. *arXiv preprint arXiv:2401.05561*, 2024.

[35] Yu Tian, Xiao Yang, Jingyuan Zhang, Yinpeng Gongye, and Kai Wang. Evil geniuses: Delving into the safety of LLM-based agents. *arXiv preprint arXiv:2311.11855*, 2024.

[36] Amos Tversky and Daniel Kahneman. Judgment under uncertainty: Heuristics and biases. *Science*, 185(4157):1124–1131, 1974.

[37] Boxin Wang, Weixin Chen, Hengzhi Pei, Chulin Xie, Min-Je Kang, Chejian Zhang, Chaowei Xu, Zidi Xiong, Ritik Dutta, Rylan Schaeffer, Sang T. Truong, Simran Steinber, Ziniu Mallen, Jonathan Tow, Sheng Tan, and Bo Li. DecodingTrust: A comprehensive assessment of trustworthiness in GPT models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2024.

[38] Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. Jailbroken: How does LLM safety training fail? In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2024.

[39] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.

[40] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xin Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

[41] Yifan Wu, Zhengyu Huang, Chaowei Xiao, Chang Liu, and Cho-Jui Hsieh. AgentAttack: Attacking LLM agents via adversarial environment construction. *arXiv preprint arXiv:2405.02956*, 2024.

[42] Jiaqi Xue, Mengxin Zheng, Yue Hua, Yifei Shu, Xun Fang, and Zhiming Li. BadRAG: Identifying vulnerabilities in retrieval augmented generation of large language models. *arXiv preprint arXiv:2406.00083*, 2024.

[43] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.

[44] Jingwei Yi, Yueqi Xie, Bin Zhu, Keegan Hines, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. Benchmarking and defending against indirect prompt injection attacks on large language models. *arXiv preprint arXiv:2312.14197*, 2023.

[45] Wang Yuntao, Xiao Yisong, and Liu Xiao. AI agent traps: Understanding and mitigating environmental attacks on AI agents. *SSRN Electronic Journal*, 2025. SSRN 6372438.

[46] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated LLM agents. *arXiv preprint arXiv:2403.02691*, 2024.

[47] Zhexin Zhang, Leqi Lei, Lindong Wu, Rui Sun, Yongkang Huang, Chong Long, Xiao Liu, Xuanyu Lei, Jie Tang, and Minlie Huang. SafetyBench: Evaluating the safety of large language models with multiple choice questions. *arXiv preprint arXiv:2309.07045*, 2024.

[48] Zexuan Zhong, Ziqing Huang, Alexander Wettig, and Danqi Chen. Poisoning retrieval corpora by injecting adversarial passages. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[49] Andy Zou, Zifan Wang, J. Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.

[50] Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models. *arXiv preprint arXiv:2402.07867*, 2024.

A Full Scenario Catalog

A complete listing of all 22 trap scenarios with their adversarial payloads, success criteria, and paper references is available in the supplementary materials.

B Extended Statistical Tables

Per-scenario, per-model raw metrics with full confidence intervals are provided in the supplementary JSON artifacts.

C Artifact Availability

The complete testbed, including all 22 adversarial scenarios, 7 mitigation modules, experiment harness, and analysis scripts, is available at: https://github.com/aviraldua93/agent-traps-lab. All experiment data is archived at https://doi.org/XX.XXXX/zenodo.XXXXXXX.